Lab 4 using your specific repository, layout, and networking assumptions. This version **treats Local subnets as private networks** and **treats the remote server / inter-site network as a 10.10.x.x "internet/public network"** for the purposes of NAT and routing logic. It also assumes the repository hierarchy:

```
None
automation/
├── mockup-infra/
└── week02-stateless-stateful/
    └── phase1-mockup/
```

We will produce:
✅ A revised **high-fidelity lab plan** ✅ **Detailed addressing & topologies** ✅ **Cisco-themed network section** ✅ **IaC / VS Code section** ✅ **Troubleshooting and engineering reflection** ✅ A complete **README.md** ready to be committed to the repo

---

- นายพัชรพล กองแก้ว 673380415-5 Sec 4
- นายกฤษฎา นามมนต์เทียน 673380388-2 Sec 4
- นายวงศธร ธน.ยอด 673380425-2 Sec 4
- นายปรีชา ศรีหนองบัว 653380335-1 Sec4
- นายศุภกิตติ์ ฟันเฟือย 673380427-8 Sec 4
- นายดรัณภพ สุริเตอร์ 673380402-4 Sec 4

🧪 LAB 4 Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

🎯 LAB GOALS

1. **Network Setup:** Create two local labs (A & B) connected via a simulated "public/internet" network (10.10.0.0/16)
2. **NAT + Routing:** Configure static routes and NAT so local traffic reaches remote services
3. **Application Deployment:** Use `mockup-infra` from your repository to deploy stateless and stateful APIs
4. **Testing:** Compare how sessions behave locally vs across the simulated internet

---

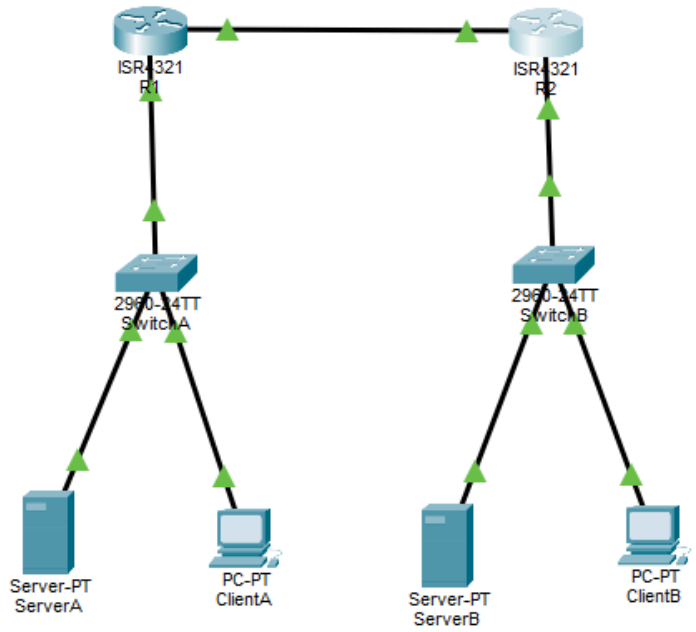
👥 TEAM BREAKDOWN (5–6 students)

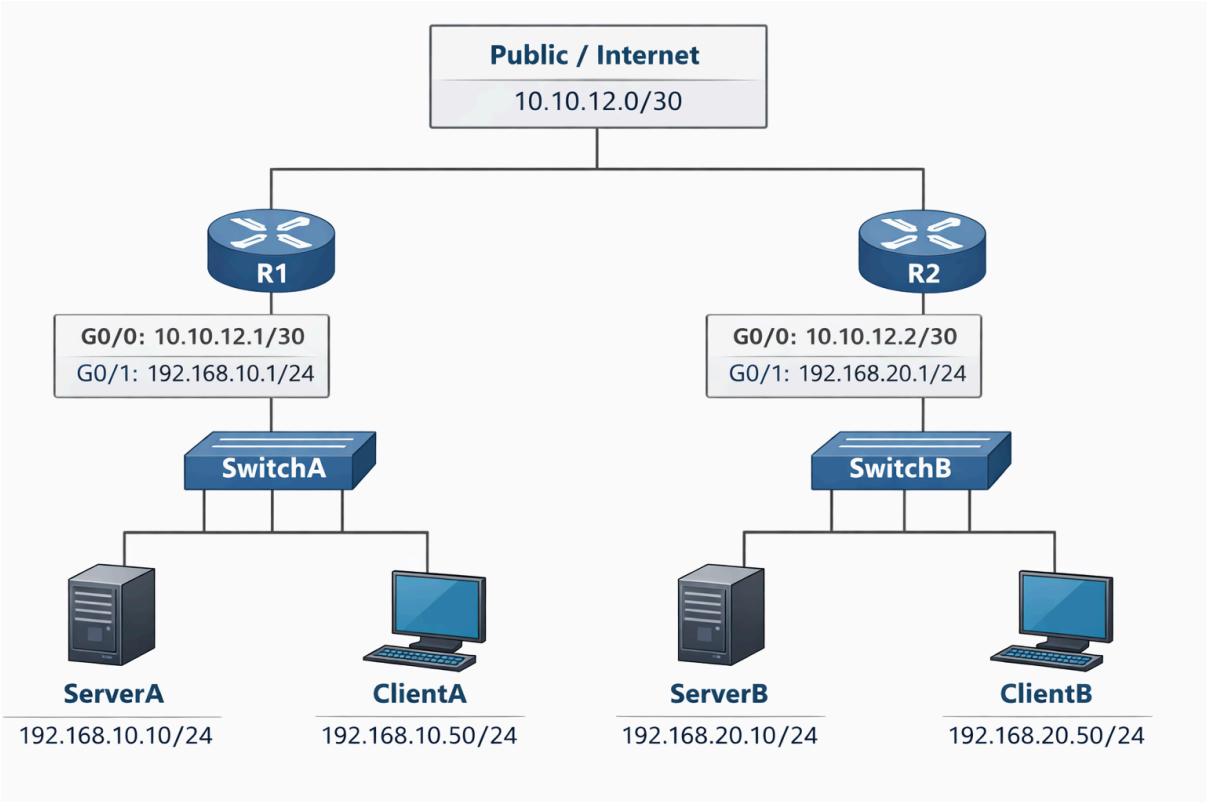
Role	Count	Focus
Network Architects	2	Router configs, Cisco CLI
IaC/DevOps	2	VS Code, mockup-infra
QA/Test	1–2	Server/Client scripting, session behavior

🗺️ UPDATED NETWORK TOPOLOGY

```python
                10.10.12.0/30 "Public/Internet"
            ┌──────────────────────────┐
            |                          |
        R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
            |                          |
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
            |                          |
        SwitchA                      SwitchB
            |                          |
 ServerA (mockup)            ServerB (mockup)
 ClientA                         ClientB
```

We treat 10.10.12.0/30 as a **simulated internet** for the purposes of NAT + public reachability.





📍 ADDRESS TABLE

Component	Interface	IP	Mask	Role
R1	G0/0	10.10.12.1	/30	Public
R1	G0/1	192.168.10.1	/24	LAN A Gateway
ServerA	eth0	192.168.10.10	/24	Mockup infra A
ClientA	eth0	192.168.10.50	/24	Tester
R2	G0/0	10.10.12.2	/30	Public
R2	G0/1	192.168.20.1	/24	LAN B Gateway
ServerB	eth0	192.168.20.10	/24	Mockup infra B
ClientB	eth0	192.168.20.50	/24	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A)

```
None
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

🧱 R2 CONFIGURATION (LAN B)

```
None
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

🧪 ROUTING & NAT VERIFICATION

On R1 & R2:

```
None
show ip route
show ip nat translations
```

R1

Physical Config CLI Attributes

```
R1(config-if)# exit
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

R1(config)#ip route 192.168.20.0 255.255.255.0 10.10.12.2
R1(config)#access-list 10 permit 192.168.10.0 0.0.0.255
R1(config)#ip nat inside source list 10 interface g0/0/0 overload
R1(config)#end
R1#write memory
%SYS-5-CONFIG_I: Configured from console by console

Building configuration...
[OK]
R1#
R1#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2

R1#ping 10.10.12.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.12.2, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 0/0/0 ms

R1#ping 192.168.10.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.10.1, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/2/4 ms

R1#ping 10.10.12.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.12.2, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/0 ms

R1#ping 192.168.20.10

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.20.10, timeout is 2 seconds:
.!!!!
Success rate is 80 percent (4/5), round-trip min/avg/max = 0/1/2 ms
```

Top

R2

Physical Config CLI Attributes

IOS Command Line Interface

```
%LINK-5-CHANGED: Interface GigabitEthernet0/0/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/1, changed state to up

R2(config)#ip route 192.168.10.0 255.255.255.0 10.10.12.1
R2(config)#access-list 10 permit 192.168.20.0 0.0.0.255
R2(config)#ip nat inside source list 10 interface g0/0/0 overload
R2(config)#end
R2#write memory
%SYS-5-CONFIG_I: Configured from console by console

Building configuration...
[OK]
R2#
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0/0
L       10.10.12.2/32 is directly connected, GigabitEthernet0/0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0/1
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0/1

R2#show ip nat translations
R2#
```

Copy Paste

Top

From ClientA:

None

```
ping 192.168.20.10  ← Remote server reachable
```

```
C:\>ping 192.168.20.10

Pinging 192.168.20.10 with 32 bytes of data:

Reply from 192.168.20.10: bytes=32 time=18ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126
Reply from 192.168.20.10: bytes=32 time=10ms TTL=126
Reply from 192.168.20.10: bytes=32 time<1ms TTL=126

Ping statistics for 192.168.20.10:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 18ms, Average = 7ms

C:\>
```

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (mockup-infra/README.md)

We will now set up mockup infra using the paths you specified:

```
None
automation/mockup-infra
automation/week02-stateless-stateful/phase1-mockup
```

🛠️ CLONE REPOปรีชา ศรีหนองบัว

```
Shell
git clone https://github.com/boonsup/automation.git
cd automation
```

🧰 VS CODE — OPEN PROJECT

In VS Code, open the `automation` folder.

📄 CREATE SERVER-SPECIFIC ENV FILES

ServerA (`ServerA.env`)

```
None
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

```
# Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

ServerB (`ServerB.env`)

```
None
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

```
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30
```

🚀 DEPLOY MOCKUP-INFRA

PowerShell (Windows Admin)

```
None
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful
```

Bash (Linux)

```
Shell
cd mockup-infra
chmod +x scripts/*.sh
./scripts/init-env.sh
python manage.py deploy --profile week02-stateless-stateful
```

```
ckup-infra$ python3 manage.py test

🧪 Testing Layer 7 Endpoints:
==================================================

📡 Testing: Public Web (L7 HTML)
   URL: http://localhost:8080/
   Status Code: 200
   ✅ Success
   Response: <!DOCTYPE html> <html lang="en"> <head>     <
meta charset="UTF-8">     <title>Public Internet Server</t
itle>     <style>         body {             fo...

📡 Testing: Intranet API Status (L7 JSON)
   URL: https://localhost:8443/status
   Status Code: 200
   ✅ Success
   Response: {
      "authenticated": true,
      "internal_ip": "172.19.0.3",
      "layer": "L7 Application",
      "network": "private_net",
      "protocol": "HTTP/1.1",
      "service": "intranet_api",
      "status": "operational",
      "timestamp": "2026-03-11T15:42:48.657636"
}

📡 Testing: Intranet API Data (L7 JSON)
   URL: https://localhost:8443/data
   Status Code: 201
   ✅ Success
   Response: {
      "message": "Data received in secure zone",
      "network": "private_net",
      "received": {
            "test": "data"
      },
      "timestamp": "2026-03-11T15:42:48.674443"
}

📡 Testing: Intranet API Config (L7 JSON)
   URL: https://localhost:8443/config
   Status Code: 200
```

```
📡 Testing: Intranet API Config (L7 JSON)
   URL: https://localhost:8443/config
   Status Code: 200
   ✅ Success
   Response: {
      "api_version": "v1",
      "environment": "secure_intranet",
      "features": [
            "encryption",
            "audit",
            "rate_limit"
      ],
      "network_isolation": true
}

📡 Testing: Health Check
   URL: http://localhost:8080/health
   Status Code: 200
   ✅ Success
   Response: {
      "service": "public_app",
      "status": "healthy",
      "network": "public_net",
      "ip": "172.18.0.3"
}

📊 Test Results: 5/5 passed
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4/automation/mo
ckup-infra$
```

🟡 VERIFY CONTAINERS

None

```
podman ps
```

```
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4/automation/mockup-infra$ podman ps
CONTAINER ID  IMAGE                                       COMMAND               CREATED        STATUS        PORTS                                       NA
MES
35835fb3e9f2  localhost/mockup-infra_public_app:latest    python app.py         6 minutes ago  Up 6 minutes                                              mo
ckup-public-web
1d28daaf6525  localhost/mockup-infra_intranet_api:latest  python api.py         6 minutes ago  Up 6 minutes                                              mo
ckup-intranet-api
08018199c571  docker.io/library/nginx:alpine              nginx -g daemon o...  6 minutes ago  Up 6 minutes  0.0.0.0:8080->80/tcp, 0.0.0.0:8443->443/tcp  mo
ckup-gateway
896b2864f7cb  localhost/mockup-infra_mime-server:latest   python server/mai...  6 minutes ago  Up 6 minutes                                              mi
me-server
c3963c4c714d  localhost/stateless-api:v2                  node start-statel...  6 minutes ago  Up 6 minutes                                              st
ateless-api
81207e005c2d  localhost/stateful-api:v2.1                 node start-statef...  6 minutes ago  Up 6 minutes                                              st
ateful-api
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4/automation/mockup-infra$
```

Should show:

Container	Port
stateless-api	3000
stateful-api	3001

🧪 PART 3 — TESTING STATELESS vs STATEFUL (week03-stateless-stateful/phase1-mockup/README.md)

⚫ LOCAL TESTS (Private LAN)

From ClientA:

```shell
curl http://192.168.10.10:3000/info
```

→ Stateless

```
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4$ curl http://localhost:3000/info
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","timestamp":"2026-03-11T
16:00:18.081Z","requestCount":1,"clientId":"anonymous","randomValue":0.23436401212571123,"message":"I have no memory of previous re
quests. Each request is independent.","note":"This is a stateless server - I don't remember you from previous requests."},"timestam
p":"2026-03-11T16:00:18.081Z"}teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4$
```

From ClientA:

```shell
curl -X POST http://192.168.10.10:3001/session
```

```
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4$ curl -X POST http://localhost:3001/session \p://localhost:3001/session \
  -H "Content-Type: application/json" \
  -d '{"userId": "user001"}'
{"status":"success","message":"Session created successfully","data":{"session":{"id":"a26aebdb-97cb-438a-bd62-c0730be2c88f","userId
":"user001","createdAt":"2026-03-11T16:05:42.138Z","expiresAt":"2026-03-11T16:20:42.138Z"},"user":{"id":"user001","name":"Test User
 1","email":"user1@example.com"},"note":"Session created. Include Session-ID header in subsequent requests to maintain state."},"ti
```

Store the returned `Session-ID`, then:

```shell
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"
```

```
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4$ curl http://localhost:3001/session -H "Session-ID: a26aebdb-97cb-438a-bd62-c0730be2c8
8f"TWORK/LAB4$ curl http://localhost:3001/session -H "Session-ID: a26aebdb-97cb-438a-bd62-c0730be2c88f"
{"status":"success","message":"Session information","data":{"session":{"id":"a26aebdb-97cb-438a-bd62-c0730be2c88f","userId":"user00
1","createdAt":"2026-03-11T16:05:42.138Z","lastAccessed":"2026-03-11T16:07:31.854Z","expiresAt":"2026-03-11T16:20:42.138Z","data":{
}},"note":"Session retrieved. This server remembers your session across requests."},"timestamp":"2026-03-11T16:07:31.854Z"}teayear@
```

🔵 REMOTE TESTS (Via Simulated Internet)

From ClientA:

```none
curl http://10.10.12.2:3000/info
```

That will target **ServerB stateless** — NAT + routing should deliver

```
Linux-Teayear:~/Teayear/NETWORK/LAB4$ curl httcurl http://localhost:3000/info
{"status":"success","message":"Stateless server information","data":{"server":"Stateless Mock Server v1.0","timestamp":"2026-03-11T
16:09:25.001Z","requestCount":2,"clientId":"anonymous","randomValue":0.8133332250373295,"message":"I have no memory of previous req
uests. Each request is independent.","note":"This is a stateless server - I don't remember you from previous requests."},"timestamp
```

Session Test:

```none
curl -X POST http://10.10.12.2:3001/session
```

```
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4$ curl -X POST http://localhost:3001/session   -H "Content-Type: application/json"   -d
 '{"userId": "user001"}'
{"status":"success","message":"Session created successfully","data":{"session":{"id":"f3ff9e1e-f7ad-4107-9fa0-957e5bcf9eea","userId
":"user001","createdAt":"2026-03-11T16:13:23.714Z","expiresAt":"2026-03-11T16:28:23.714Z"},"user":{"id":"user001","name":"Test User
 1","email":"user1@example.com"},"note":"Session created. Include Session-ID header in subsequent requests to maintain state."},"ti
```

→ ServerB returns a session ID

Then:

```none
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"
```

```
teayear@Linux-Teayear:~/Teayear/NETWORK/LAB4$ nux-Thttp://localhost:3001/session -H "Session-ID: f3ff9e1e-f7ad-4107-9fa0-957e5bcf9e
ea"
{"status":"success","message":"Session information","data":{"session":{"id":"f3ff9e1e-f7ad-4107-9fa0-957e5bcf9eea","userId":"user00
1","createdAt":"2026-03-11T16:13:23.714Z","lastAccessed":"2026-03-11T16:13:36.140Z","expiresAt":"2026-03-11T16:28:23.714Z","data":{
}},"note":"Session retrieved. This server remembers your session across requests."},"timestamp":"2026-03-11T16:13:36.140Z"}teayear@
```

Check session persists across NAT

⚠️ TROUBLESHOOTING

❗ Cannot Reach Remote

- Gateway misconfiguration
- NAT ACL incorrect
- Wrong default gateway on clients

Check:

```
None
show ip route
ping 10.10.12.2 from R1
```

❗ Sessions Lose State

- Missing header
- Wrong container port
- Firewall blocking port 3001

🧠 ENGINEERING REFLECTION

1. **Stateless services don't depend on prior data**
2. **Stateful services require session headers**
3. NAT shouldn't affect logical session identity

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.
```

```
## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```